

AEGIS402

Technical Security Documentation

A Local-First Security Boundary for Autonomous x402 Payments

Version 0.1 • Experimental Prototype Prepared from the supplied project brief and the publicly accessible Aegis402 repository security documentation. College: Dr. Akhilesh Das Gupta Institute of Professional Studies

Security position: this document intentionally describes implemented guarantees and their limits rather than future architecture.



The banner features a dark background with green and white text and icons. On the left, there is a green shield icon with a padlock and circuitry, followed by the text 'AEGIS402' in large white and green letters, and 'SECURE. VERIFY. SETTLE. PROTECT.' in smaller white letters. To the right, a list of team members is shown with icons: a person for 'TEAM LEADER WINZAA', a group of people for 'TEAM MEMBER PARI', and a graduation cap for 'COLLEGE ADGIPS'. Below this, a 'PROBLEM STATEMENT' section includes a target icon, the text 'Autonomous AI agents using x402 for payments face critical security risks due to weak verification, context binding, and settlement integrity. Design a local, fail-closed security middleware that validates every 402 response before allowing settlement or resource access.', and two buttons labeled 'SET-2' and 'PROMPT - 3'.

AEGIS402
SECURE. VERIFY. SETTLE. PROTECT.

TEAM LEADER
WINZAA

TEAM MEMBER
PARI

COLLEGE
ADGIPS

PROBLEM STATEMENT

SET-2 PROMPT - 3

Autonomous AI agents using x402 for payments face critical security risks due to weak verification, context binding, and settlement integrity. Design a local, fail-closed security middleware that validates every 402 response before allowing settlement or resource access.

1. Executive Summary

Aegis402 is a local security middleware for x402 payment flows. Its purpose is not to replace x402 verification or blockchain settlement, but to introduce an additional security boundary between an incoming paid request and resource release. The prototype evaluates whether the payment context, policy, freshness, authorization window, nonce state, execution-cost estimate, and settlement response remain consistent with the request being served.

The motivation is a gap between payment validity and payment integrity. In an autonomous payment flow, a payment proof may be cryptographically valid while still being presented in the wrong resource context, reused concurrently, or evaluated against requirements that have changed since the original authorization. Aegis402 therefore asks a narrower security question: is this exact payment acceptable for this exact resource and request under the current policy and state?

The implementation is deliberately prototype-oriented. Version 0.1 uses a deterministic MockX402Adapter by default for local execution and attack simulation, while exposing an adapter boundary for integration with a live x402 facilitator. The repository explicitly states that Aegis402 does not replace x402 verification or blockchain settlement. It also documents important limitations: nonce atomicity is process-local, the execution guard compares a caller-provided estimated cost rather than measuring gas or compute, and the local hash chain is tamper-evident rather than tamper-proof.

This document presents the implementation as it exists rather than as a future security architecture. Claims about stronger distributed guarantees, independent on-chain amount verification, external ledger anchoring, or production-grade multi-replica atomicity are treated as future work rather than current capabilities.

2. Problem Statement and Security Motivation

x402 turns HTTP 402 into a programmatic payment flow for APIs, content, and machine-to-machine services. In the normal flow, a client requests a protected resource, receives payment requirements in a 402 response, creates a payment payload, and sends it back for verification and settlement. The protocol is intentionally modular: verification and settlement may be delegated to a facilitator.

That architecture creates a security boundary that is easy to overlook. The HTTP request, payment requirements, signed authorization, verification result, settlement result, and eventual resource release are related state, but they are not automatically one indivisible application-level operation. Security therefore depends on preserving the relationships between them.

Recent security research has identified concrete classes of x402 risk. Ling et al. describe cross-resource substitution and concurrency-related logic flaws, while Wang et al. report attacks including Free Shopping, Asset Theft, Service Denial, and Gas Abuse in facilitator-mediated deployments. These findings motivate controls that bind payment context to the resource and request, re-check state close to settlement, and prevent duplicate execution.

Aegis402 addresses this narrower problem: a valid payment proof should not be treated as sufficient evidence that the current request is safe to execute. The gateway instead evaluates multiple predicates before resource release.

The central security distinction is:

Payment validity asks whether a payment satisfies the payment protocol.

Payment integrity asks whether that payment is still the correct payment for this resource, merchant, amount, network, asset, request state, and execution context.

Aegis402 is designed around the second question.

3. Architecture and Request Flow

The Aegis402 gateway sits in front of payment verification and resource release. The incoming request and its payment state enter the gateway. The gateway evaluates the resource and merchant context, payment policy, freshness and authorization validity, execution constraints, and nonce state. Verification and settlement are then delegated to the configured x402 adapter. A security event is recorded after the decision.

Conceptually, the request path is:

Client request→ payment-required state→ Aegis402 gateway→ context binding→ policy evaluation→ freshness / authorization checks→ nonce reservation and idempotency→ x402 adapter verification→ settlement→ settlement-response consistency checks→ allow resource release or reject→ append security event

The repository separates the security middleware from the x402 integration through an adapter boundary. MockX402Adapter is the default implementation and is explicitly intended for deterministic local execution and tests. X402FacilitatorAdapter provides the integration boundary for the official x402 Python SDK and a facilitator endpoint.

This separation is important to the prototype's claims. Aegis402 is a policy and integrity layer around x402; it is not itself the blockchain verifier, wallet, facilitator, or settlement engine.

The main implementation areas are organized around the middleware, tests, demo material, dashboard, and documentation. The project uses FastAPI for its HTTP interface and can be run locally with Uvicorn. The README also identifies attack tests for concurrent nonce reservation and ledger integrity.

4. Threat Model

The primary assets are: access to the protected resource, the payment authorization state, the request/resource binding, nonce state, settlement-result metadata, and the integrity of the local security record.

The relevant attacker capabilities are application-level manipulation and misuse of payment state rather than compromise of the underlying blockchain cryptography. An attacker may attempt to reuse a valid payment in a different resource context, race multiple requests using the same nonce, exploit stale payment requirements, exceed configured spending policy, induce abnormal execution cost, or cause the application to accept an inconsistent settlement response.

Aegis402 is intended to reduce the attack surface at the application boundary. It does not claim to protect against a compromised host, a compromised Python runtime, a malicious process with unrestricted filesystem access, compromise of cryptographic primitives, or every possible facilitator failure.

Threats are best understood as invariant violations:

1. Context invariant — the payment must match the resource and merchant context for which it was authorized.
2. Policy invariant — the amount, endpoint, merchant, network, and related constraints must remain within configured policy.
3. Freshness invariant — the payment requirements and authorization window must still be acceptable when the request is processed.
4. Single-use invariant — a nonce must not allow multiple callers to reach settlement within the protected process.
5. Settlement invariant — the settlement response must be consistent with the authorized payment to the extent the response exposes those fields.
6. Audit-integrity invariant — recorded security events should form a chain in which modification of an existing record is detectable.

The model intentionally distinguishes prevention from detection. For example, the hash-chained ledger can reveal modification of existing records, but it cannot prevent an operator with write access from replacing the entire file with a new internally consistent chain.

5. Security Mechanisms

Context binding is the first enforcement layer. The documented implementation rejects a payment when the presented resource, merchant, pay-to address, amount, scheme, network, or asset differs from the values against which the payment was authorized. This is the control most directly aimed at cross-resource payment substitution.

Policy enforcement adds local application constraints. Configured amount ceilings and endpoint, merchant, and network restrictions are evaluated before verification and settlement. This gives the application a policy layer that is independent of whether the underlying payment payload is otherwise valid.

Freshness has two related meanings in the prototype. First, the gateway can reject stale payment requirements based on local server state. Second, where the payment scheme provides a signed authorization validity window, that signed window is checked. The security documentation explicitly notes that the local freshness value is server state, while the signed validity window is the part committed by the payer when supported by the scheme.

The authorization window is therefore not treated as a universal blockchain-time guarantee. It is an application-level acceptance condition that must be evaluated at the point of processing.

The execution guard is intentionally modest. The prototype does not measure actual gas or compute consumption. Instead, it receives an estimated cost from the caller and compares that estimate with a configured ceiling. This can demonstrate policy enforcement and abnormal-cost handling in the prototype, but it should not be described as a trusted gas-metering mechanism.

The default decision philosophy is fail closed: when a required security condition fails, the gateway rejects the request rather than proceeding to settlement.

6. Atomicity, Idempotency and Settlement Integrity

The concurrency problem is particularly important because payment verification and settlement can be separated in time. If two callers reach the same state concurrently, both may appear valid before either has consumed the relevant nonce.

Aegis402 introduces nonce reservation and idempotency protection. In the current implementation, a nonce guard uses an in-memory set protected by a thread lock. Within one running process, this allows only one caller to reach settlement for a given nonce.

This is a meaningful prototype property, but it is deliberately not described as distributed atomicity. Multiple worker processes or replicas do not share the in-memory reservation state. A production deployment would require shared transactional state or an equivalent coordination mechanism.

Settlement integrity is the final consistency check. A failed settlement, missing transaction hash, or returned payer or network mismatch causes rejection. The exact strength of this check depends on which fields are available in the settlement response.

An important limitation is that the settlement response alone cannot independently establish the amount actually transferred on chain when that fact is not exposed as an interoperable field. If an application needs to prove the settled amount from chain state, it must independently inspect the relevant transaction.

Therefore the prototype's settlement layer should be understood as response-consistency validation, not a complete independent blockchain accounting engine.

7. Threat Ledger and Tamper Evidence

Aegis402 records security decisions in a local hash-chained ledger. Each event is linked to prior state so that modifications to existing records can be detected. The chain is persisted across process restarts.

The ledger is useful for three purposes. First, it creates an audit trail for accepted and rejected decisions. Second, it gives attack demonstrations a concrete artifact that can be inspected after an event. Third, hash chaining makes accidental or unauthorized modification of an existing record detectable during integrity verification.

The correct security terminology is tamper-evident, not tamper-proof.

A process that already has write access to the ledger can replace the ledger with a newly constructed internally consistent chain. The current design does not anchor the chain to an external immutable system. Stronger provenance would require an external anchor, independent storage, signatures, or another trust mechanism.

This distinction is important because the ledger's purpose in v0.1 is local evidence rather than decentralized consensus.

The same implementation-first discipline applies to the rest of the system. The ledger does not make Aegis402 a decentralized security network. The prototype is local-first and process-oriented. A future distributed threat-intelligence design could build on the event format, but such a network is outside the demonstrated v0.1 implementation.

The dashboard and attack demonstrations can use the ledger as a human-readable explanation of why a request was blocked or allowed, while the chained structure provides an additional integrity signal for the recorded history.

8. Attack Scenarios and Validation

The project is designed around demonstrable attack scenarios rather than a claim of exhaustive security coverage. The security argument should therefore be read as: the prototype implements specific guards against specific classes of state and logic errors, and the tests demonstrate those guards under controlled conditions.

The most important scenario categories are:

Cross-resource substitution: a payment authorized for one resource or context is presented for another. Expected result: context binding rejects the mismatch.

Policy violation: the amount, endpoint, merchant, or network falls outside configured policy. Expected result: the request is rejected before settlement.

Stale requirements: payment requirements are no longer current. Expected result: freshness validation blocks the request.

Authorization-window violation: a signed validity interval, where supplied by the scheme, is no longer acceptable. Expected result: the gateway rejects the payment.

Replay / duplicate execution: the same nonce is submitted again. Expected result: nonce reservation and idempotency prevent a second caller from reaching settlement within the protected process.

Concurrent race: two callers attempt the same nonce at the same time. Expected result: the process-local guard permits only one caller to reach settlement.

Execution-cost abuse: a caller supplies an estimated cost above the configured ceiling. Expected result: the execution guard rejects the request.

Settlement inconsistency: settlement fails, lacks the required transaction hash, or returns inconsistent payer/network information. Expected result: the gateway rejects the request.

Ledger tampering: an existing event is modified. Expected result: hash-chain verification detects the modification.

These tests demonstrate the behavior of the prototype under its stated assumptions. They should not be presented as proof that all x402 attacks are prevented. In particular, live facilitator behavior, multi-process coordination, and independent on-chain amount verification require additional testing and infrastructure.

9. Dashboard and Live Demonstration

The project includes a dashboard and a local attack runner intended to make the security model observable rather than purely theoretical. The repository identifies `dashboard/` as the local HTTP interface and `demo/` as the location for the local attack runner and demonstration material.

For a judge-facing demonstration, the recommended sequence is:

1. Start from the normal payment flow and show an allowed request.
2. Trigger a context-binding attack and show that the payment is rejected because the request context no longer matches the authorization.
3. Trigger a replay or concurrent nonce scenario and show that the nonce guard prevents duplicate settlement within the process.
4. Trigger a freshness or authorization-window failure and show the security decision.
5. Trigger a settlement inconsistency and show that resource release is blocked.
6. Open the event/ledger view and show the recorded decision.
7. If the dashboard exposes ledger verification, modify a test record and demonstrate that the hash chain detects the change.

The live deployment supplied with the project is intended as a demonstration surface. The documentation should not imply that the hosted demo constitutes production deployment or live-funds validation unless the deployed adapter and environment have independently been verified.

The most effective judge-facing message is therefore not "Aegis402 makes x402 secure." It is: "Aegis402 adds a local, fail-closed integrity boundary that checks whether a payment is still appropriate for the exact resource and execution context before settlement and resource release."

That statement matches the implementation and leaves room for the limitations described elsewhere in this document.

10. Deployment, Repository, Limitations and Roadmap

Aegis402 is currently an experimental v0.1 prototype. The repository documents a local FastAPI interface, a dashboard, tests, demo material, and an x402 adapter boundary. The standard local development flow creates a Python virtual environment, installs requirements, and starts the FastAPI application with Uvicorn. The README exposes the dashboard and OpenAPI documentation locally.

The deployment model is intentionally simple: place the middleware in the request path before resource release, configure the desired policy, and select the appropriate x402 adapter. The MockX402Adapter should remain a test and demonstration adapter. The facilitator adapter should be validated against the exact x402 Python SDK version used in deployment.

Current limitations must remain visible in any security claim:

- No independent security audit has been performed.
- The prototype must not be used to protect production payment flows or real funds.
- Atomicity is process-local and does not coordinate replicas.
- The execution guard does not measure actual gas or compute.
- The mock adapter does not perform live cryptographic verification.
- Settlement response consistency does not by itself prove the on-chain amount transferred.
- The local ledger is tamper-evident rather than tamper-proof.
- Stronger distributed provenance requires external anchoring or equivalent infrastructure.

A sensible roadmap is incremental. First, replace process-local nonce state with shared transactional coordination. Second, integrate and test the live facilitator path against pinned SDK versions. Third, add independent chain-state inspection where settlement amount verification is required. Fourth, strengthen ledger provenance through external anchoring or signed append-only storage. Finally, a future version could introduce privacy-preserving, cryptographically verifiable threat intelligence between independent payment nodes.

Conclusion: Aegis402's contribution is a security boundary, not a replacement payment protocol. It makes the relationship between payment, resource, request, policy, freshness, execution state, and settlement explicit and enforceable at the application layer. That is the core security argument the prototype can credibly demonstrate today.

Appendix A — Implementation Traceability

The following traceability map is intended to keep the documentation aligned with the repository rather than with aspirational architecture.

Context binding → gateway checks resource, merchant, pay-to, amount, scheme, network and asset.

Policy → configured amount ceilings plus endpoint, merchant and network restrictions.

Freshness → stale requirement rejection and signed authorization-window checks where supplied.

Atomicity / idempotency → process-local nonce reservation using an in-memory set and thread lock.

Execution guard → caller-supplied estimated cost compared with a configured ceiling.

Settlement integrity → rejection on failed settlement, missing transaction hash, or payer/network mismatch where those fields are available.

Threat ledger → local hash-chained event records persisted across process restarts.

Adapter boundary → MockX402Adapter for deterministic local testing; X402FacilitatorAdapter for integration with the x402 Python SDK and facilitator.

This mapping should be updated whenever implementation changes. A security document should never retain a stronger claim after the corresponding implementation has been removed or weakened.

Appendix B — Reproducibility and Reviewer Checklist

A technical reviewer should be able to answer the following questions without relying on marketing language:

- What exact component sits between the incoming request and resource release?
- Which payment fields are bound to the request and merchant context?
- Which policy fields are configurable?
- What is considered stale, and where does freshness state come from?
- What happens if two requests use the same nonce concurrently?
- Does the nonce guard work across processes or replicas?
- Does the prototype perform live cryptographic verification?
- Can settlement response data prove the amount actually transferred?
- What exactly does the ledger integrity check prove?
- What happens if an attacker can rewrite the entire ledger?
- Which attack demonstrations are deterministic?
- Which demonstrations depend on the mock adapter?
- Which claims require a live facilitator or chain-state inspection?

For a hackathon submission, these questions are more valuable than adding pages of generic cybersecurity terminology. They directly connect the security argument to executable behavior.

Appendix C — References

Primary protocol and implementation references:

1. Coinbase Developer Documentation — x402 overview and payment flow. 2. x402 Foundation — open-source protocol specification and typical request/verify/settle flow. 3. Ling, Huang, Chen, Zhou, Wu and Wang (2026), "Free-Riding in the AI Economy: Demystifying Logic Flaws in x402-Enabled Payment Systems." 4. Wang, Yang, Chen, Ji and Payer (2026), "When HTTP 402 Meets the Blockchain: Risks on Emerging x402 Payments." 5. Stantchev (2026), "Hardening x402: PII-Safe Agentic Payments via Pre-Execution Metadata Filtering." 6. Aegis402 repository README and SECURITY.md.

The external research is used here to motivate the problem and terminology. The Aegis402 repository and its security policy remain the source of truth for claims about what the prototype itself implements.

Repository: <https://github.com/msWinzaa/Aegis402Demo> supplied with the project:
<https://aegis402.onrender.com/dashboard>